

Programming in Modern C++: Assignment Week 12

Total Marks : 25

Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur, Kharagpur – 721302
partha.p.das@gmail.com

October 2, 2025

Question 1

Which of the following correctly matches each smart pointer type with its ownership policy?
[MCQ, Marks 2]

- a) `std::auto_ptr` (C++03) – Shared ownership
 `std::unique_ptr` (C++11) – Shared ownership
 `std::shared_ptr` (C++11) – Exclusive ownership
 `std::weak_ptr` (C++11) – Exclusive ownership
- b) `std::auto_ptr` (C++03) – Exclusive ownership
 `std::unique_ptr` (C++11) – Exclusive ownership
 `std::shared_ptr` (C++11) – Shared ownership
 `std::weak_ptr` (C++11) – Shared ownership
- c) `std::auto_ptr` (C++03) – Shared ownership
 `std::unique_ptr` (C++11) – Exclusive ownership
 `std::shared_ptr` (C++11) – Exclusive ownership
 `std::weak_ptr` (C++11) – Shared ownership
- d) `std::auto_ptr` (C++03) – Exclusive ownership
 `std::unique_ptr` (C++11) – Shared ownership
 `std::shared_ptr` (C++11) – Shared ownership
 `std::weak_ptr` (C++11) – Exclusive ownership

Answer: b)

Explanation:

`std::auto_ptr` (C++03) and `std::unique_ptr` (C++11) provide **exclusive ownership**.
`std::shared_ptr` (C++11) and `std::weak_ptr` (C++11) follow the **shared ownership** model.

Question 2

Consider the code segment (C++11) given below.

[MCQ, Marks 2]

```
#include<iostream>
using namespace std;

template<typename T>
class Wrapper {
public:
    explicit Wrapper(T* ptr): ptr_(ptr) { }
    ~Wrapper() { delete ptr_; }
    T& operator*() const { return *ptr_; }
    T* operator->() const { return ptr_; }
    operator T*() { return ptr_; }
    operator void*() { return ptr_; }
private:
    T* ptr_;
};

void square(int* p) { *p = (*p) * (*p); }

int main(){
    Wrapper<int> wp(new int);
    *wp = 4;
    cout << *wp << " ";
    square(wp);          // LINE-1
    cout << *wp << " ";
    delete wp;           // LINE-2
    return 0;
}
```

What will be the output/error?

- a) 4 16
- b) 4 4
- c) compiler error at LINE-1: cannot convert 'Wrapper' to 'int*'
- d) compiler error at LINE-2: ambiguous default type conversion from 'Wrapper'

Answer: d)

Explanation:

Although `square(wp);` works correctly due to the implicit conversion to `int*`, the statement `delete wp;` produces a compiler error. This is because the compiler finds two possible user-defined conversions (`operator T*()` and `operator void*()`) and cannot resolve the ambiguity. Hence, it results in ambiguous default type conversion from 'Wrapper'.

Question 3

Consider the code segment (C++11) given below.

[MCQ, Marks 2]

```
#include <iostream>
#include <memory>

void update(std::shared_ptr<int>& sp) {
    sp.reset(new int(50));
}

int main() {
    auto sp1 = std::make_shared<int>(25);
    auto sp2 = sp1;
    std::weak_ptr<int> w1 = sp1;
    std::weak_ptr<int> w2 = sp2;

    sp2.reset(new int(30));
    *sp2 = 35;

    if(auto p = w1.lock())
        std::cout << *p << std::endl;
    else
        std::cout << "w1 expired" << std::endl;

    update(sp1);

    if(auto p = w2.lock())
        std::cout << *p << std::endl;
    else
        std::cout << "w2 expired" << std::endl;

    return 0;
}
```

What will be the output?

- a) 35
50
- b) w1 expired
w2 expired
- c) 25
w2 expired
- d) 25
50

Answer: c)

Explanation:

Initially, both `sp1` and `sp2` point to the integer 25, so both `w1` and `w2` observe this object. When `sp2.reset(new int(30))` is executed, `sp2` points to a new object 30, while `sp1` still owns 25. Thus, `w1.lock()` succeeds and prints 25.

Later, `update(sp1)` resets `sp1` to a new object 50, destroying the original 25. Since `w2` was still tracking the destroyed object, `w2.lock()` fails, and the second output is `w2 expired`.

Question 4

Consider the code segment (C++11) given below.

[MSQ, Marks 2]

```
#include <iostream>
#include <vector>
#include <thread>

void addVal(std::vector<int> &v){
    v.push_back(3);
}

int main(){
    std::vector<int> v;
    for(int i = 1; i <= 2; i++)
        v.push_back(i);
    std::thread t1 {&addVal, ref(v)};

    for(int x : v)
        std::cout << x << " ";
    t1.join();
    return 0;
}
```

What is/are the possible output?

- a) 1 2
- b) 1 2 3
- c) 3 1 2
- d) It prints nothing

Answer: a), b)

Explanation:

Explanation:

The vector `v` initially stores `{1, 2}`. A new thread is spawned, which adds 3 by reference. Since `push_back` always appends elements to the end, the order of elements is preserved.

- If the main thread completes the iteration before the worker thread executes, the output is: 1 2.
- If the worker thread executes before the iteration starts, the output is: 1 2 3.

Thus, only options (a) and (b) are valid. Option (c) (3 1 2) cannot occur, because `push_back` does not insert at the front. If `push_back` triggers a reallocation while the main thread is iterating, the behavior is technically **undefined** in C++, but it still does not lead to a predictable 3 1 2 ordering.

Question 5

Consider the code segment (C++11) given below.

[MCQ, Marks 2]

```
#include <iostream>
#include <memory>

void display(std::shared_ptr<int> q) {
    std::cout << "Inside display, RC = " << q.use_count() << " ";
}

int main(){
    std::shared_ptr<int> p1(new int(5));
    std::cout << "RC = " << p1.use_count() << " ";

    std::shared_ptr<int> p2 = p1;
    std::cout << "RC = " << p1.use_count() << " ";

    {
        std::shared_ptr<int> p3 = p2;
        std::cout << "RC = " << p1.use_count() << " ";
        display(p3);
        p3.reset();
        std::cout << "RC = " << p1.use_count() << " ";
    }

    p2.reset(new int(20));
    std::cout << "RC = " << p1.use_count();

    return 0;
}
```

What will be the output?

- a) RC = 1 RC = 2 RC = 3 Inside display, RC = 4 RC = 2 RC = 1
- b) RC = 1 RC = 2 RC = 3 Inside display, RC = 4 RC = 2 RC = 0
- c) RC = 1 RC = 2 RC = 3 Inside display, RC = 3 RC = 2 RC = 1
- d) RC = 1 RC = 2 RC = 2 Inside display, RC = 3 RC = 1 RC = 0

Answer: a)

Explanation:

The code execution can be explained step by step as follows:

```
// Step-1: p1 created -> RC = 1
// Step-2: p2 created as copy of p1 -> RC = 2
// Step-3: p3 created as copy of p2 -> RC = 3
// Step-4: display(p3) is called by value -> temporary copy made -> RC = 4
// Step-5: On exit from display(), temporary copy destroyed -> RC = 3
// Step-6: p3.reset() releases p3 -> RC = 2
// Step-7: p2.reset(new int(20)) detaches p2 from old object -> RC = 1
```

Question 6

Consider the code segment (C++11) given below.

[MCQ, Marks 2]

```
#include <iostream>
#include <thread>
#include <future>

int main () {
    auto alpha = _____;    //LINE-1
    auto beta  = _____;    //LINE-2

    std::thread th1 = std::thread([&] { alpha.set_value(99); });
    std::thread th2 = std::thread([&] {
        if(beta.get() % 2 == 0) std::cout << "even";
        else std::cout << "odd";
    });

    th1.join();
    th2.join();
    return 0;
}
```

Identify the appropriate option to fill in the blanks at LINE-1 and LINE-2, which ensure the output is odd.

- a) LINE-1: `std::promise<int>()`
LINE-2: `alpha.get_future()`
- b) LINE-1: `std::future<int>()`
LINE-2: `alpha.get_future()`
- c) LINE-1: `std::promise<int>()`
LINE-2: `std::future<int>()`
- d) LINE-1: `std::future<int>()`
LINE-2: `alpha.get_promise()`

Answer: a)

Explanation: At LINE-1, a `std::promise<int>` object can be created with `std::promise<int>()`. At LINE-2, the associated `std::future<int>` object can be obtained using `alpha.get_future()`. The thread `th1` sets the value in the promise, and `th2` retrieves it from the future. Since the value is 99, the output is odd.

Question 7

Consider the program (in C++11) given below.

[MCQ, Marks 2]

```
#include <iostream>
#include <functional>

int calculate(int a, int b, const int& c, int d) {
    return (a * b - c + d);
}

int main() {
    using namespace std::placeholders;
    int y = 4;
    auto func = std::bind(calculate, _2, y, std::cref(y), _1);
    y = 6;
    std::cout << func(3, 2);
    return 0;
}
```

What will be the output?

- a) 4
- b) 5
- c) 6
- d) 8

Answer: b)

Explanation:

The bind expression assigns arguments as follows:

- `_1` is assigned to the last parameter `d`, so `d = 3`.
- `_2` is assigned to the first parameter `a`, so `a = 2`.
- `b = y = 4` (captured by value at bind time).
- `c = std::cref(y)` refers to the current value of `y`, which is 6 at execution time.

Thus, the function call evaluates as:

$$\text{calculate}(a = 2, b = 4, c = 6, d = 3) = 2 * 4 - 6 + 3 = 8 - 6 + 3 = 5$$

Hence, the output is 5.

Question 8

Consider the following code segment (in C++11).

[MSQ, Marks 2]

```
#include <iostream>
#include <thread>
#include <mutex>

struct buffer{
    int count = 0;
};

struct queue{
    int count = 0;
};

std::mutex mtx_buf;
std::mutex mtx_que;
static buffer buf;
static queue que;

void producer(int n) {
    std::unique_lock<std::mutex> lck1(mtx_buf);
    std::unique_lock<std::mutex> lck2(mtx_que);
    buf.count += n;
    que.count += n;
    std::cout << buf.count << ", " << que.count << std::endl;
}

void consumer(int n) {
    std::unique_lock<std::mutex> lck1(mtx_que);
    std::unique_lock<std::mutex> lck2(mtx_buf);
    buf.count += n;
    que.count += n;
    std::cout << buf.count << ", " << que.count << std::endl;
}

int main(){
    std::thread t1{producer, 3};
    std::thread t2{consumer, 7};
    t1.join();
    t2.join();
    return 0;
}
```

Which of the following **cannot** be TRUE about the output of the above program?

a) It generates output as:

3, 3
10, 10

b) It generates output as:

7, 7
10, 10

c) It generates output as:

7, 7

3, 3

d) It results in deadlock

Answer: c)

Explanation:

The code in `producer` and `consumer` uses `unique_lock` to acquire both mutexes. Therefore, possible outcomes are: 3, 3 and 10, 10, or 7, 7 and 10, 10.

If one thread locks `mtx_buf` and the other locks `mtx_que`, they may wait for each other, resulting in deadlock.

However, the output 7, 7

3, 3 cannot occur.

Intentionally kept as MSQ

Question 9

Consider the code segment (C++11) given below.

[MSQ, Marks 2]

```
#include <iostream>
#include <thread>
#include <mutex>

std::mutex mtx_acc;
struct account{ int balance {15}; };

void withdraw(account& acc, std::string user, int amt){
    mtx_acc.lock();
    if(amt > acc.balance){
        std::cout << "withdrawal by " << user << " : failed, ";
    }
    else{
        acc.balance -= amt;
        std::cout << "withdrawal by " << user;
        std::cout << " : balance remaining = " << acc.balance << ", ";
    }
    mtx_acc.unlock();
}

int main(){
    account acc;
    std::thread t1 {withdraw, std::ref(acc), "Alice", 6};
    std::thread t2 {withdraw, std::ref(acc), "Bob", 5};
    std::thread t3 {withdraw, std::ref(acc), "Charlie", 7};
    t1.join(); t2.join(); t3.join();
    return 0;
}
```

Which of the following outputs is/are **not** possible? (think logically)

- a) withdrawal by Alice : balance remaining = 9
withdrawal by Charlie : balance remaining = 2
withdrawal by Bob : failed
- b) withdrawal by Charlie : failed
withdrawal by Alice : balance remaining = 2
withdrawal by Bob : failed
- c) withdrawal by Charlie : balance remaining = 8
withdrawal by Alice : balance remaining = 2
withdrawal by Bob : failed
- d) withdrawal by Charlie : balance remaining = 8
withdrawal by Alice : balance remaining = 2
withdrawal by Bob : balance remaining = 1

Answer: b), d)

Explanation:

Since the entire code of `withdraw` is executed after calling `mtx_acc.lock()`, the operations on `acc` happen mutually exclusively. Therefore, the outputs in options b) and d) cannot occur logically.

Programming Questions

Question 1

Consider the following program (in C++11).

- Fill in the blank at LINE-1 with appropriate header to overload function operator.
- Fill in the blank at LINE-2 to invoke the functor NthFibo() asynchronously.
- Fill in the blank at LINE-3 to receive the output from functor NthFibo().

The program must satisfy the sample input and output.

Marks: 3

```
#include <iostream>
#include <future>
using namespace std;

struct NthFibo{
    const int n;
    NthFibo(const int& n) : n(n) { }

    ----- { //LINE-1
        if(n <= 1) return n;
        long long a = 0, b = 1, c;
        for(int i = 2; i <= n; i++){
            c = a + b;
            a = b;
            b = c;
        }
        return b;
    }
};

long long findNthFibo(int n){
    ----- ; //LINE-2
    return ----- ; //LINE-3
}

int main() {
    int n;
    cin >> n;
    cout << findNthFibo(n) << endl;
    return 0;
}
```

Public 1

Input: 10

Output: 55

Public 2

Input: 15

Output: 610

Private

Input: 25

Output: 75025

Answer:

```
LINE-1: long long operator()()
LINE-2: auto a = async(NthFibo(n))
LINE-3: a.get()
```

Explanation:

At LINE-1, we overload the function call operator to make `NthFibo` a functor: `long long operator()()`.

At LINE-2, the functor is invoked asynchronously using `std::async`: `auto a = async(NthFibo(n))`.

At LINE-3, the result of asynchronous execution is retrieved using: `a.get()`.

Thus, the program computes the n^{th} Fibonacci number asynchronously using a functor.

Question 2

Consider the program below (in C++11), which implements a simple smart pointer for a 'Record' class.

- Fill in the blank at LINE-1 with appropriate header and initializer list for the copy constructor.
- Fill in the blank at LINE-2 with appropriate header to overload dereferencing operator.
- Fill in the blank at LINE-3 with appropriate header to overload indirection operator.

The program must satisfy the given test cases.

Marks: 3

```
#include <iostream>

class Record {
public:
    Record() = default;
    explicit Record(int v) : val_(v) {}
    explicit Record(const Record& r) : val_(r.val_) {}
    void setValue(int v) { val_ = v; }
    friend std::ostream& operator<<(std::ostream& os, const Record& r) {
        os << r.val_ << " ";
        return os;
    }
private:
    int val_;
};

template <typename T>
class MySmartPtr {
public:
    explicit MySmartPtr(T* ptr) : ptr_(ptr) {}
    ----- { // LINE-1: copy constructor
        other.ptr_ = nullptr;
    }
    ~MySmartPtr() { if(ptr_ != nullptr) delete ptr_; }
    ----- { return *ptr_; } // LINE-2: dereferencing
    ----- { return ptr_; } // LINE-3: indirection
private:
    T* ptr_;
};

int main() {
    int x, y;
    std::cin >> x >> y;
    MySmartPtr<Record> rp(new Record(x));
    std::cout << *rp;
    rp->setValue(y);
    std::cout << *rp;
    MySmartPtr<Record> rp2 = rp;
    std::cout << *rp2;
    return 0;
}
```

Public 1

Input: 7 15

Output: 7 15 15

Public 2

Input: 3 9

Output: 3 9 9

Private

Input: 12 -5

Output: 12 -5 -5

Answer:

LINE-1: `MySmartPtr(MySmartPtr& other) : ptr_(other.ptr_)`

LINE-2: `T& operator*() const`

LINE-3: `T* operator->() const`

Explanation:

At LINE-1, the copy constructor header and initializer list can be written as:

`MySmartPtr(MySmartPtr& other) : ptr_(other.ptr_)`

At LINE-2, the header to overload the dereferencing operator is:

`T& operator*() const`

At LINE-3, the header to overload the indirection operator is:

`T* operator->() const`

Question 3

Consider the following program in C++14 to represent a doubly linked list (generic type), which allows adding items at the end of the list. Complete the program as per the instructions given below.

- Fill in the blank at LINE-1 with appropriate statements to traverse the list in forward direction.
- Fill in the blank at LINE-2 with appropriate statements to traverse the list in backward direction.

The program must satisfy the sample input and output.

Marks: 3

```
#include <iostream>
#include <memory>

template<typename T>
class DList;

template<typename U>
class Node {
public:
    Node(U _val) : val(_val), next(nullptr) {}
    friend DList<U>;
private:
    U val;
    std::shared_ptr<Node<U>> next;
    std::weak_ptr<Node<U>> prev;
};

template<typename T>
class DList {
public:
    DList() = default;
    void addEnd(const T& item) {
        std::shared_ptr<Node<T>> n = std::make_shared<Node<T>>(item);
        if (first == nullptr) {
            first = last = n;
        } else {
            n->prev = last;
            last->next = n;
            last = n;
        }
    }

    void biTraverse() {
        for (_____) // LINE-1
            std::cout << t->val << " ";
        std::cout << std::endl;
        for (_____) // LINE-2
            std::cout << p->val << " ";
    }
}
```

```

private:
    std::shared_ptr<Node<T>> first = nullptr;
    std::shared_ptr<Node<T>> last = nullptr;
};

int main() {
    DList<int> lst;
    int n, a;
    std::cin >> n;
    for (int i = 0; i < n; i++) {
        std::cin >> a;
        lst.addEnd(a);
    }
    lst.biTraverse();
    return 0;
}

```

Public 1

Input:
4
10 20 30 40
Output:
10 20 30 40
40 30 20 10

Public 2

Input:
3
1 2 3
Output:
1 2 3
3 2 1

Public 3

Input:
1
5
Output:
5
5

Private

Input:
5
10 20 30 40 50
Output:
10 20 30 40 50
50 40 30 20 10

Answer:

LINE-1: `std::shared_ptr<Node<T>> t = first; t != nullptr; t = t->next`

LINE-2: `std::weak_ptr<Node<T>> t = last; auto p = t.lock(); t = p->prev`

Explanation:

Since in class `node`, `next` is a `shared_ptr`, the for loop for forward traversal must be:

`std::shared_ptr<node<T>> t = first; t != nullptr; t = t->next)`

Since in class `node`, `prev` is a `weak_ptr`, the backward traversal can be done compactly using a weak pointer inside the for loop:

`for(std::weak_ptr<node<T>> t = last; auto p = t.lock(); t = p->prev) { ... }`

Here, `t` starts from the last node, `p = t.lock()` gets a `shared_ptr` to safely access the node, and `t = p->prev` moves backward in the list. This achieves a safe and complete backward traversal.